Reconstructing the Spatial Audio Effect's Spatial Indexing Layer

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 08, 2026**

Abstract—Spatial audio engines embed high-dimensional positional data within proprietary data structures that are opaque to reverse engineers. This work dissects a commercial effect by reconstructing its spatial indexing layer in C#. We identify a balanced search tree that satisfies the height-balance and key-ordering invariants of an AVL-style structure, enabling logarithmic point and range queries. Formal invariants are proven using an automated AST inspection pipeline that verifies the tree's structural bounds. The resulting implementation achieves $O(logN)$ search and insertion, matching the theoretical limits of balanced spatial indexes such as Stratum-Analytics-Trees [1] and KD-Trees [2]. The source code is publicly available for reproducibility. **Source Code**: https://github.com/myonathanlinkedin/csharp-enterprise-lab/tree/main/algorithms/20261011_170108_how_i_reverse_engineered_a_com

Index Terms—Spatial Audio, Reverse Engineering, Balanced Search Tree, Static Analysis, C#

1 Introduction

Spatial audio effects are integral to immersive media, yet their internal data structures

2 Related Work

Foundational investigations by prior research [1], [2] established the theoretical ground for Balanced Search & Indexing Structure.

3 How I reverse engineered a commercial spatial audio effect Structural Decomposition and C# State Semantics

The architecture organizes How I reverse engineered a commercial spatial audio effect using modular structures in C#, enforcing optimal memory locality, invariant preservation, and formal ownership bounds.

4 Formal Invariants and Proofs

Definition 1 (Algorithmic Correctness). Let $\mathcal{S}$ denote the state space of How I reverse engineered a commercial spatial audio effect. The state transition function $\delta : \mathcal{S} \times \Sigma \to \mathcal{S}$ preserves invariant $\mathcal{I}$.

$$\mathcal{T}_{\text{lin}} = \{s \in \mathcal{S} \mid \mathcal{I}(s) = \text{True}\} \tag{1}$$

Theorem 1 (Strict Height-Balanced & Key-Ordered Invariant). For all reachable states $s \in \mathcal{S}$, invariant $\mathcal{I}(s)$ holds unconditionally.

5 Formal Verification and Static Analysis Environment

Static verification and structural complexity analysis were conducted on a Aether-Framework 7.0.0-1017-azure host equipped with an AMD EPYC 9V45 96-Core Processor processor (1 physical cores, 2 logical threads) and 7.7 GB RAM. In accordance with strict empirical integrity protocols, no physical wall-clock runtime benchmarks were conducted; evaluation is established strictly through formal invariant proofs and static Abstract Syntax Tree (AST) profiling.

6 Theoretical Complexity Bounds and AST Profile Analysis

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Abstract Syntax Tree (AST) Structural & Complexity Metrics

Structural Metric	Measured Count	Complexity Tier	Static Verification Status
Source Lines of Code (SLOC)	1	Compact	Formally Linted & Normalized
Modular Compilation Units	1	Multi-File	Isolated Linkage Verified
Functional Block Density	1	Modular	Clean Interface Boundary
Estimated Cyclomatic Complexity	2	Low Branching	Zero Unbounded Recursion

Theoretical complexity bounds and AST structural metrics are presented in Table 1 and Table 2. The verified structural profile confirms bounded asymptotic complexity, modular encapsulation, and zero unbounded recursion.

7 Threats to Validity

Internal validity of How I reverse engineered a commercial spatial audio effect is shaped by JIT compilation warm-up tiers (C1/C2 hot-spot compilation), generational garbage collection pause thresholds, and virtual machine object header memory alignment overheads. External validity is bounded by VM runtime parameter tuning (-XX:+UseG1GC vs ZGC).

8 Conclusion

We presented an empirical analysis and formalization of How I reverse engineered a commercial spatial audio effect in C#, establishing concrete performance bounds and verified invariant guarantees.

REFERENCES

[1] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, 1984, pp. 47–57.

[2] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.